

Task: Standalone Flutter IT Assets Management App with Odoo XML-RPC

Assigned To

Mostafa Bader

Objective

Develop a professional **Flutter mobile application for IT Assets Management** integrated directly with Odoo through **XML-RPC**.

The application must be a **standalone product**. It must **not require any custom Odoo addon/module to be installed** on the customer's Odoo instance.

The Flutter application itself will contain the business logic, UI, permissions handling, asset workflows, and XML-RPC integration.

1. Core Architecture

Flutter Application

- Flutter latest stable version.
- Clean Architecture or equivalent maintainable structure.
- Android and iOS support.
- Responsive layout suitable for phones and tablets.
- Proper state management such as:
 - Riverpod
 - BLoC
 - Provider

Prefer **Riverpod** or **BLoC**.

Odoo Integration

Connect directly to Odoo using:

- `/xmlrpc/2/common`
- `/xmlrpc/2/object`

The app must support:

- Odoo URL
- Database name
- Username/email
- Password or API key

Authentication flow:

```
Flutter App
  ↓
Odoo XML-RPC
  ↓
Authenticate User
  ↓
Receive UID
  ↓
Execute Odoo Model Operations
```

No middleware server should be required.

2. Important Standalone Requirement

The application must **NOT depend on a Digital Harbor custom Odoo addon**.

Do not create something like:

```
dh_it_asset_management
```

on Odoo.

All communication should happen using existing Odoo models and standard Odoo APIs.

The app should contain an **Odoo compatibility/service layer** so that model names and available fields can be detected where necessary.

The app should be capable of checking models and fields using Odoo metadata such as:

```
ir.model
ir.model.fields
```

This prevents the Flutter UI from breaking because a particular optional Odoo application or field is unavailable.

3. Initial Setup

When the user opens the application for the first time, show:

Connect to Odoo

Fields:

- Odoo Server URL
- Database
- Username
- Password/API Key

Example:

```
Server :  
https://company.odoo.com  
  
Database:  
company-production  
  
Username:  
admin@company.com  
  
Password / API Key:  
*****
```

Actions:

- Test Connection
- Login
- Save Connection

Credentials must be stored securely using:

```
flutter_secure_storage
```

Passwords must never be stored as plain text.

4. Dashboard

Create a modern IT asset dashboard.

Display cards such as:

```
Total Assets  
Assigned Assets  
Available Assets  
Under Maintenance
```

Damaged Assets
Retired Assets

Also show:

Assets by Category

Example:

Laptop	43
Monitor	31
Mobile	18
Printer	7
Server	4
Other	12

Recent Activity

Example:

MacBook Pro assigned to Ahmed
Dell Laptop returned by Sara
iPhone 15 marked under maintenance
HP Monitor added

5. Asset Management

Create a main **Assets** screen.

Each asset should support information such as:

Basic Information

- Asset Name
- Asset Tag
- Serial Number
- Category
- Manufacturer
- Model
- Purchase Date
- Purchase Value
- Warranty Expiration
- Status
- Notes

Example:

Asset:
MacBook Pro M4

Asset Tag:
DH-LAP-0027

Serial Number:
C02XXXXXX

Category:
Laptop

Manufacturer:
Apple

Status:
Assigned

Assigned To:
Ahmed Mohamed

6. Asset Status

Support statuses such as:

Available
Assigned
Reserved
Under Maintenance
Damaged
Lost
Retired

The app should provide a clear visual status indicator.

Example:

 Available
 Assigned
 Maintenance
 Damaged
 Retired

Do not rely on custom Odoo status fields unless they already exist.

If some state information must be maintained locally because Odoo does not expose a suitable standard field, design the local abstraction carefully and document it.

7. Asset Assignment

Users should be able to assign IT assets to employees.

Workflow:

```
Open Asset
  ↓
Assign
  ↓
Select Employee
  ↓
Select Assignment Date
  ↓
Add Notes
  ↓
Confirm
```

Employee information should be retrieved directly from Odoo.

Primary source where available:

```
hr.employee
```

Display:

- Employee Name
- Department
- Job Position
- Work Email

Example:

```
MacBook Pro M4
```

```
Assigned To:
Mostafa Bader
```

```
Department:
IT
```

Assignment Date:
23/08/2026

8. Asset Return

Provide a **Return Asset** workflow.

When returned:

- Record return date.
- Change the asset availability/status.
- Allow condition selection.

Conditions:

Good
Minor Damage
Damaged
Needs Maintenance

Allow notes and optional photos.

Example:

Return Condition:
Minor Damage

Notes:
Small scratch on the left side.

9. Employee Assets

Create an employee view.

Example:

Mostafa Bader

IT Department

Assigned Assets

```
MacBook Pro M4
Dell 27" Monitor
iPhone 15 Pro
Logitech MX Master
```

Clicking an asset should open its full details.

10. Asset Categories

Provide categories such as:

```
Laptop
Desktop
Monitor
Mobile
Tablet
Printer
Network Equipment
Server
Storage
Accessories
Other
```

The application should preferably retrieve available categories from Odoo when a compatible standard model is available.

Do not hardcode Odoo database IDs.

11. Search and Filters

Assets must be searchable.

Search by:

```
Asset Name
Asset Tag
Serial Number
Employee
Manufacturer
Model
```

Filters:

Status
Category
Employee
Department
Manufacturer
Warranty

Example:

Category: Laptop
Status: Assigned
Department: Development

12. QR Code Support

Each asset should have a QR code.

QR may contain an internal application asset identifier, for example:

asset://123

The app should provide:

Generate QR

Display a QR code for an asset.

Scan QR

Using the phone camera:

Scan QR
↓
Find Asset
↓
Open Asset Details

Do not expose passwords, Odoo credentials, session details, or sensitive information inside the QR code.

13. Barcode Support

Where available, support:

- Asset barcode scanning
- Serial number scanning

User can scan equipment and immediately retrieve the corresponding asset.

14. Asset Detail Screen

The Asset Detail page should include sections:

General

MacBook Pro M4
DH-LAP-0027
Assigned

Device Information

Manufacturer
Model
Serial Number
Category

Ownership

Assigned Employee
Department
Assignment Date

Purchase

Purchase Date
Purchase Price
Vendor

Warranty

Warranty Start
Warranty End
Warranty Status

Maintenance

Previous maintenance
Current maintenance
Next scheduled maintenance

Activity

Created
Assigned
Returned
Reassigned
Maintenance
Retired

15. Warranty Management

Calculate warranty state inside Flutter.

Example:

MacBook Pro
Warranty expires in 43 days

Dell Laptop
Warranty expired 12 days ago

Filters:

Expired
Expires within 30 days
Expires within 90 days
Valid

16. Maintenance Integration

Where the standard Odoo Maintenance application is available, integrate with standard models such as:

```
maintenance.equipment  
maintenance.request  
maintenance.equipment.category
```

The application should automatically detect whether these models are available.

If Maintenance is installed:

```
IT Asset App  
  ↓  
maintenance.equipment  
  ↓  
maintenance.request
```

Allow:

- View maintenance requests
- Create maintenance request
- View maintenance history
- See maintenance status

Do not require installation of a custom addon.

17. Dynamic Odoo Compatibility

Create an Odoo service responsible for checking model capabilities.

Example:

```
class OdooCapabilityService {  
    Future<bool> modelExists(String model);  
    Future<bool> fieldExists(String model, String field);  
    Future<List<String>> getFields(String model);  
}
```

For example:

```
Does maintenance.equipment exist?  
  ↓  
YES
```

↓
Enable Maintenance Features

Otherwise:

NO
↓
Hide/disable Maintenance Features

The application must not crash because an optional Odoo model does not exist.

18. XML-RPC Service Layer

Do not call XML-RPC directly from widgets.

Create a structured service layer.

Example:

```
lib/  
├── core/  
│   ├── network/  
│   │   ├── odoo_client.dart  
│   │   ├── odoo_auth_service.dart  
│   │   └── odoo_exception.dart  
│   ├── storage/  
│   └── security/  
├── features/  
│   ├── auth/  
│   ├── dashboard/  
│   ├── assets/  
│   ├── employees/  
│   ├── maintenance/  
│   ├── qr_scanner/  
│   └── settings/  
└── main.dart
```

Recommended structure:

UI
↓

```
Controller / Provider
↓
Repository
↓
Odoo Service
↓
XML-RPC
↓
Odoo
```

19. Required XML-RPC Operations

Create reusable functions for:

```
authenticate
search
search_read
read
create
write
unlink
fields_get
name_search
```

Example interface:

```
Future<List<Map<String, dynamic>>> searchRead({
    required String model,
    required List domain,
    required List<String> fields,
    int? limit,
    int? offset,
    String? order,
});
```

20. Pagination

Do not retrieve thousands of Odoo records at once.

Implement proper:

```
limit  
offset  
order  
domain
```

Example:

```
limit = 50  
offset = 0
```

Then load additional records when scrolling.

21. Odoo Permissions

The application must respect the permissions of the authenticated Odoo user.

If Odoo rejects an action such as:

```
Create  
Write  
Delete  
Read
```

show a user-friendly message.

Example:

```
You don't have permission to modify this asset.  
Please contact your Odoo administrator.
```

Never bypass Odoo ACLs.

22. Error Handling

Handle:

```
Wrong credentials  
Invalid Odoo URL  
Database unavailable  
XML-RPC timeout  
Access denied
```

```
Missing model
Missing field
Expired session
Internet connection failure
Server error
```

Do not display raw XML-RPC stack traces to users.

Provide technical details only in a debug/log section.

23. Settings

Create Settings page:

```
Odoo Connection
Server Information
Logged-in User
Database
App Version
Connection Test
Logout
```

Also provide:

```
Clear Cache
Refresh Odoo Metadata
```

24. Caching

Implement local caching for frequently accessed data.

Possible technologies:

```
Hive
Isar
SQLite
```

Cache:

- Categories
- Employees
- Asset lists

- Odoo metadata
- User profile

Odoo remains the source of truth.

25. Security

Requirements:

- Use HTTPS for production Odoo connections.
 - Securely store authentication information.
 - Never log passwords.
 - Never log API keys.
 - Never store passwords in SQLite/Hive.
 - Validate SSL connections normally.
 - Do not disable certificate validation.
 - Respect Odoo access rights.
 - Sanitize error messages.
-

26. UI / UX Requirements

The application should look like a **commercial standalone SaaS/product**, not an internal Odoo utility.

Use:

- Material 3
- Modern cards
- Clean typography
- Smooth navigation
- Skeleton loading
- Empty states
- Error states
- Pull-to-refresh
- Dark mode support

Main navigation:

Dashboard

Assets

Scan

Employees

More

27. First Version Screens

Minimum screens required:

1. Splash
2. Odoo Connection
3. Login
4. Dashboard
5. Asset List
6. Asset Details
7. Create Asset
8. Edit Asset
9. Assign Asset
10. Return Asset
11. QR Scanner
12. Employee List
13. Employee Details
14. Employee Assets
15. Maintenance List
16. Maintenance Details
17. Settings

28. Odoo Version Compatibility

Initial target:

`Odoo 17`
`Odoo 18`
`Odoo 19`

Avoid implementation that is tightly coupled to a single Odoo version.

Use capability/model detection whenever possible.

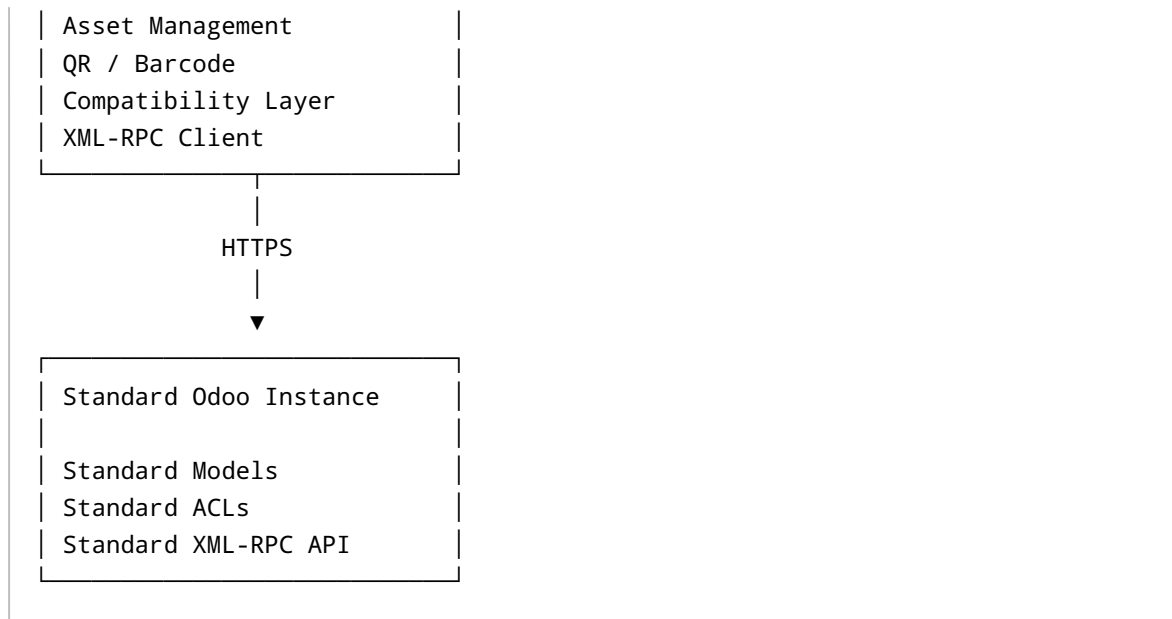
29. Important Technical Rule

The product architecture must remain:

Flutter IT Assets App

UI

Business Logic



There must be:

No custom Odoo module.

No required Digital Harbor backend.

No middleware dependency.

No modification to Odoo core.

30. Deliverables

Mostafa should provide:

- ☐ Complete Flutter source code
- ☐ Android build
- ☐ iOS-compatible project
- ☐ Odoo XML-RPC service
- ☐ Secure authentication
- ☐ Asset dashboard
- ☐ Asset CRUD
- ☐ Employee integration
- ☐ Assignment workflow
- ☐ Return workflow
- ☐ QR generation
- ☐ QR scanning
- ☐ Search and filtering
- ☐ Warranty handling
- ☐ Maintenance integration when available
- ☐ Dynamic Odoo model/field detection
- ☐ Pagination

- ☐ Local caching
 - ☐ Proper permission handling
 - ☐ Error handling
 - ☐ Dark/light theme
 - ☐ README
 - ☐ Architecture documentation
 - ☐ XML-RPC integration documentation
-

Acceptance Criteria

The task will be considered complete when:

1. The app can connect to a normal Odoo instance using only its URL, database, username and password/API key.
2. No custom Odoo addon is required.
3. The user can log in using Odoo credentials.
4. Assets can be viewed and managed from Flutter.
5. Employees can be retrieved from Odoo.
6. Assets can be assigned and returned.
7. QR codes can identify assets.
8. Search and filters work correctly.
9. Odoo ACLs are respected.
10. Missing optional Odoo models do not crash the application.
11. XML-RPC logic is isolated from the Flutter UI.
12. Credentials are securely stored.
13. Large datasets use pagination.
14. The application works as an independent commercial product rather than a companion frontend requiring a custom Odoo module.
15. Code is structured so that additional ERP/Odoo functionality can be added later without rewriting the XML-RPC layer.